

APIx — Viva Voce Preparation (Master Edition, Verified Against Code)

Team CodeCrew | Smart India Hackathon 2026 | PS ID: SIH26056 *Development of a Real-time Airfare Price Index for India through Automated Web Scraping of Airline and OTA Portals for Augmentation of the Consumer Price Index (CPI)*

How this document was built: this merges your original Q&A set with the uploaded PDF, but every fact in it was checked directly against the current codebase before being kept. A few things in the PDF turned out to be outdated (pre-dating your 30-route expansion) or inaccurate even before that — those corrections are folded directly into the relevant Q&As below rather than listed separately. The **DGCA Watchdog feature was missing entirely** from the PDF and has been added as its own section (Section 8).

1. Project Overview & Motivation

Q1. What problem does your project solve, in one sentence? APIx builds a real-time, high-frequency Airfare Price Index from live-scraped airline and OTA fares, and uses it to augment India's official Consumer Price Index (CPI), whose airfare sub-component currently relies on slow, manually-collected counter quotes.

Q2. Why does the airfare component of CPI need "augmentation"? MoSPI's eSankhyiki system collects the Transport CPI's airfare sub-index through manual, counter-collected price quotes. Three specific weaknesses: (1) **Lag** — roughly a 45-day reporting delay versus what a live index can offer; (2) **Coverage** — it misses dynamic, revenue-managed online pricing that can swing 200–400%, even though online sales now dominate ticket purchases; (3) **Frequency** — it's a monthly snapshot, not a continuous series, so it can't capture short-term shocks like a fuel spike or a festival demand surge.

Q3. Who is the end user / consumer of this system? Primarily NSO and RBI, who use CPI and its sub-components for inflation tracking and monetary policy. We've also built a direct line to **DGCA's Tariff Monitoring Unit** via the watchdog module (Section 8) — a regulatory, not just statistical, consumer. Secondary users: aviation-sector analysts, airlines, and OTAs for competitive benchmarking.

Q4. Why is airfare specifically important enough to warrant its own index? Air transport sits inside the Transport & Communication group of CPI (8.59% of urban CPI; airfare itself is

~0.41% of the headline basket). It's unusually volatile compared to most CPI components — fuel surcharges, dynamic pricing, and seasonal spikes — so a lagging or unrepresentative airfare reading distorts the accuracy of the broader Transport CPI disproportionately to its own small weight.

Q5. What is the project's full official title? "Development of a Real-time Airfare Price Index for India through Automated Web Scraping of Airline and Online Travel Aggregator Portals for Augmentation of the Consumer Price Index (CPI)."

2. System Architecture

Q6. Describe the end-to-end architecture. A modular pipeline of independent packages: `scrapers/` (raw fare collection), `pipeline/` (cleaning & decomposition), `index/` (index computation, eSankhyiki augmentation, and the watchdog), `database/` (SQLite persistence of raw/clean quotes, daily index, DGCA benchmarks, route weights), `backtest/` (validation against DGCA), `api/` (FastAPI service), `dashboard/` (9-tab Streamlit app).

Q7. Why modular instead of one monolithic script? Each layer can be developed, tested, and modified independently — if an airline changes its site layout, only the scraper needs fixing; the index engine, schema, and API are unaffected. It also enabled focused unit testing per module (27 tests across 7 files).

Q8. What's the tech stack, and why each piece? Python (data/scraping ecosystem) · FastAPI (async REST API with free OpenAPI docs) · Streamlit + Plotly (fast interactive dashboard without separate frontend work) · SQLite (zero-setup, sufficient at current data volumes) · `requests` for the (currently partial) live-attempt scraping path, with a calibrated synthetic pricing engine as the dominant data source — Playwright and Scrapy exist in the codebase as scaffolding for a future live-scraping upgrade, not as active dependencies today (neither is even in `requirements.txt`) · pandas/NumPy/SciPy (cleaning, index math, correlation stats) · pytest (27-test suite) · Docker (reproducible deployment).

Q9. Why SQLite rather than PostgreSQL/MySQL for a "real-time" system? At current scale (tens of thousands of quotes, 90 days of daily rows across up to 30 routes), SQLite handles the read/write load with zero setup overhead. For production/national-scale ingestion, migrating to PostgreSQL would be the natural next step — the database layer is abstracted, so this swap wouldn't require touching scraping or index-computation code.

Q10. How is the project deployed / reproduced? Containerized with a `Dockerfile` and `docker-compose.yml`, plus a `docker-entrypoint.sh` and a systemd timer for scheduled runs. It's also been run behind an **Nginx** reverse proxy in front of the FastAPI/Streamlit services, and was deployed to **AWS** for a live demo instance during development.

3. Data Collection (Scraping Layer)

Q11. Which sources do you scrape? Five airlines — IndiGo (62% market share), Air India (14%), Air India Express (8%), Akasa Air (5%), SpiceJet (4%) — and five OTAs — MakeMyTrip (₹349 convenience fee), EaseMyTrip (₹0, their USP), Yatra (₹399), Cleartrip, Ixigo.

Q12. How do you ensure the scraping is ethical/legal? `base_scraper.py` enforces a **robots.txt compliance check** and **per-domain rate limiting with randomized jitter and exponential back-off** before any request is made, plus **user-agent rotation**.

`scheduler.py` spreads requests over time rather than bursting them.

Q13. What happens when a site blocks scraping or changes layout? Rather than attempting to defeat a block or CAPTCHA, the request hands off to `synthetic_engine.py` — a calibrated revenue-management-style dynamic pricing simulator that keeps the pipeline continuously operational without ever violating a site's terms of service. This is a deliberate design choice: compliance over completeness. **Precise current-state detail:** this fallback pattern is fully implemented for the airline scraper (which does attempt one real request first). The OTA scraper doesn't attempt a live request at all in the current build — it goes to the synthetic engine directly, by design, pending live OTA integration.

Q14. Isn't the synthetic engine just "fake data"? It's a calibrated simulator, not arbitrary random numbers — it mimics realistic RM dynamics (distance-based fare curves, lead-time and day-of-week multipliers). **Be accurate about today's balance if asked directly:** it is currently the dominant data source across the pipeline, not a rare continuity measure — the OTA and Playwright scraper paths are synthetic-only today, and the airline scraper's one live attempt (to a plausible but unconfirmed endpoint) will typically fall back against real production sites too. The backtest validation compares the *computed index* (built from whatever mix of live/synthetic data the pipeline produced) against DGCA's independently-published benchmark — that comparison is real and meaningful regardless of the input mix, but don't claim the underlying quotes are predominantly "real scraped data" today. Say plainly: the architecture for live scraping is real and ethical; full live coverage is the next milestone, not the current state.

Q15. Why Playwright instead of simple `requests` /BeautifulSoup scraping? Many airline/OTA booking flows render fares via client-side JavaScript calls — a static HTTP GET wouldn't see the actual price, which is why Playwright (a real headless browser capable of intercepting the underlying network calls) was chosen as the intended production approach and scaffolded into `playwright_scraper.py`. **Be precise if asked to demo this:** in the current build, that class's actual browser-automation code is commented out — it checks whether the `playwright` package is importable, logs the result, and then unconditionally

returns synthetic data either way. `playwright` isn't even in `requirements.txt` yet. The *design rationale* for choosing Playwright is real and defensible; the *live execution* of it isn't wired up yet. Say exactly that if pressed, rather than implying it's actively driving a browser today.

Q16. What exactly is captured per fare quote? Route, search date, travel date (determining the advance-purchase window), source (airline/OTA), and the fare breakup where available — feeding directly into the decomposition step.

4. Data Cleaning & Fare Decomposition

Q17. Why can't raw scraped data go straight into the index? It contains duplicates (same fare captured across overlapping runs or re-listed by multiple OTAs), missing/sold-out gaps, and statistical outliers that would distort an index if included naively.

Q18. How are outliers handled, and why "stratified" IQR? A stratified Interquartile Range filter — computed **within each route × advance-purchase-window bucket separately**, not globally. A ₹15,000 fare might be a wild outlier on a short route but completely normal on a long trunk route; a single global threshold would over-filter cheap routes or under-filter expensive ones.

Q19. How are sold-out flights handled? Modeled as a distinct data state, not zero-filled (which would artificially crash the average) and not silently dropped (which biases the average toward the cheaper seats that are still available). Instead, imputed using the 85th percentile of comparable flights in the same route-window.

Q20. What is "fare decomposition" and why bother? Every fare is split into base fare, GST (5% economy / 12% business, per CBIC's published slabs for air transport), UDF (₹300–480, airport-specific — **18 airports** now individually configured with their own rates), PSF (₹91–110), and OTA convenience fee (₹0–399). This matters because GST rates and airport fees change independently of the airline's actual pricing decisions — mixing them in would contaminate the index with tax/fee-policy noise rather than pure market price movement.

Q21. What does deduplication actually remove? Redundant captures of the same flight-date-fare instance seen across overlapping scraper runs or re-listed identically by multiple OTAs — the latest cleaner update explicitly **prioritizes the direct-airline-portal quote** over an OTA quote when both exist for the same flight, rather than an arbitrary ordering.

5. The Route Basket & Index Construction (Econometrics)

Q22. What index formula does APlx actually use? One production formula: the **Two-Tier Jevons-Laspeyres Hybrid Index**, explicitly labeled in code as the "Official Champion Formulation," conforming to IMF CPI Manual (2020) and Eurostat HICP guidelines for scanner/web-scraped aviation data.

- **Tier 1 (elementary/micro):** within each (route, advance-purchase-window) cell, prices are aggregated with a **Jevons geometric mean** — $\exp(\text{mean}(\ln(P)))$ — chosen because it neutralizes asymmetric surge spikes (300%+ swings) and algorithmic volatility without needing individual-transaction weights.
- **Tier 2 (corridor/macro):** those elementary Jevons prices are aggregated **across corridors using Laspeyres-style expenditure weighting**, driven by empirical DGCA annual passenger volumes — ensuring compatibility with MoSPI's official CPI 2012=100 framework.

Q23. Why not just use one classical formula throughout? A pure Laspeyres at the elementary level would need individual-transaction expenditure weights we don't have. A pure Jevons at the aggregate level would treat every route as equally important, understating a high-traffic corridor. The hybrid gets Jevons' robustness to volatility where it's needed (within-cell) and Laspeyres' real-world traffic weighting where it matters (across-route).

Q24. What about Paasche, Fisher, and pure Jevons — don't you compute those too? They exist in the codebase as **explicitly deprecated legacy stubs**, retained only for backward database compatibility — not as parallel production computations. The code comments state directly: Paasche is deprecated because it "requires real-time daily flyer counts which airlines do not disclose"; Fisher is "superseded by Two-Tier Jevons-Laspeyres Hybrid Index"; pure Jevons is deprecated because it "lacks passenger traffic volume representation." Be precise about this in the viva — claiming all four are actively computed and compared is not accurate to the current code.

Q25. What's the current route basket, and how big is it? **30 domestic corridors, covering approximately 69% of national domestic passenger traffic** — expanded from an original 6-route pilot basket. Weights are recomputed per-route as that route's annual passenger count over the sum across all 30 routes, from DGCA's own published data (cross-verified via the open India Aviation Traffic dataset).

Q26. What are the top-weighted routes now? BOM–DEL 11.51% (still the single largest corridor, but far more diluted now that the basket spans 30 routes instead of 6), BLR–DEL 8.22%, BLR–BOM 6.96%, CCU–DEL 4.89%, DEL–MAA 4.04%, and 25 further corridors ranging down to smaller regional pairs (e.g. BLR–PNQ, DEL–IXC, CCU–GAU), spanning **18 airports total** — the original 6 (Delhi, Mumbai, Bangalore, Kolkata, Hyderabad, Chennai) plus 12 newly added: Ahmedabad, Goa, Srinagar, Guwahati, Kochi, Pune, Lucknow, Patna,

Bhubaneswar, Chandigarh, Jaipur, and Varanasi.

Q27. Why expand from 6 to 30 routes — wasn't 6 "representative enough"? Six trunk routes were a reasonable pilot basket, but 30 corridors covering ~69% of national traffic is a materially stronger claim of representativeness for a CPI-relevant index — it captures regional/leisure/pilgrimage travel patterns (e.g. Varanasi, Srinagar, Guwahati) that a 6-metro-only basket would miss entirely, closer in spirit to how the actual CPI basket aims for broad representativeness rather than just the biggest cities.

Q28. What advance-purchase windows do you use, and what are the weights? T+1 (10%), T+7 (25%), T+15 (30% — the peak "modal booking" window), T+30 (25%), T+45 (10%) — approximating a realistic real-world booking-lead-time distribution for Indian domestic travel, verified directly against the `ADVANCE_WINDOW_WEIGHTS` config.

Q29. Why does the advance-purchase window matter at all? Airfares follow revenue-management dynamic pricing — the same seat costs very differently depending on how far ahead it's booked. Sampling only T+1 would bias the index toward the most volatile, most expensive end of the curve; weighting across five windows captures the real distribution of how people actually book.

6. eSankhyiki / CPI Augmentation Module

Q30. What is eSankhyiki, and how does APlx integrate with it? MoSPI's official price-statistics data system, producing the government's Transport CPI series via its own manually-collected weights (Price Statistics Division / **UN COICOP 2018** classification, division 07 "Transport", sub-group 07.3.3.1 "Domestic Passenger Air Transport"). Our `esankhyiki.py` module (`ESankhyikiManager`) ingests this official series and its weighting structure and computes an **Augmented Transport CPI** by substituting APlx's higher-frequency airfare index in place of the manually counter-collected sub-component.

Q31. How does the substitution actually work mathematically? The official Transport CPI = (non-air component × its weight) + (manual airfare index × airfare's ~4.82% share within Transport). We algebraically back out the non-air component from the published official figure, rescale APlx's own index (base 100) onto the official 2012=100 base using a calibrated base-level airfare figure (~165.0), then recompute the composite substituting APlx's scaled value for the manual airfare term — keeping every other sub-component untouched.

Q32. What does "quantifying the gap in basis points" mean? The difference between the official Transport CPI and the Augmented version, expressed in basis points (1 bps = 0.01 percentage point) — plus a further calculation of how that gap **transmits into headline**

CPI, using Transport's 8.59% weight in the urban basket. This is directly comparable to how RBI communicates inflation-target changes.

Q33. What's the "National Data Warehouse export"? A structured export (`generate_esankhyiki_feed`) formatting the augmented index, PSD parameters, full corridor coverage (all 30 routes with weights), and the 5 advance-booking horizons into eSankhyiki/MoSPI's expected ingestion schema — including the COICOP classification code, base period, and methodology label, so it can be machine-consumed by a government pipeline, not just viewed on our dashboard.

Q34. How do you reconcile a real-time daily series with the fixed 2012=100 annual base? The augmentation module explicitly rescales APIx's own index (naturally computed against its own base period) onto the official 2012=100 base before substitution, using a calibrated base-level airfare figure, so the composite output is on a consistent scale with the rest of the officially published series.

7. Validation & Backtesting

Q35. How do you know the index is accurate? A 90-day backtest comparing computed daily index values against **DGCA's own official average-fare benchmark data** across 3 monthly snapshots (June–August 2026), computed live via `scipy.stats.pearsonr` and NumPy RMSE/MAPE. Results: **Pearson $r = 0.9997$, MAPE = 2.41%, RMSE = ₹197.47** — each comfortably clearing the project's own documented acceptance thresholds ($r \geq 0.88$, $\text{MAPE} < 5.0\%$, $\text{RMSE} < ₹450.00$).

Q36. What do these three metrics mean, and why report all three? **r (0.9997)** shows near-perfect trend/direction tracking against the benchmark. **MAPE (2.41%)** is the average error size relative to the actual fare, in percentage terms — scale-independent and intuitive. **RMSE (₹197.47)** gives the error in absolute rupees and, because it squares errors before averaging, is more sensitive to occasional large misses than MAPE. Reporting all three avoids being misled by any one metric's blind spot.

Q37. What does the "+45 day reporting lead advantage" mean? The index is available daily, essentially in near real time, while official DGCA/CPI airfare figures are published with roughly a 45-day lag — confirmed directly in code (`reporting_lead_advantage_days: 45`). That head start is the core practical value proposition.

Q38. Why benchmark against DGCA rather than directly against MoSPI's CPI airfare number? DGCA publishes route-level fare and traffic data granular enough to validate our route-weighted index directly, and it's public at usable frequency. MoSPI's CPI airfare sub-index is precisely the aggregated, low-frequency number we're trying to augment —

validating against it would be circular; DGCA gives an independent, apples-to-apples ground truth at route level.

Q39. A correlation of 0.9997 seems almost too good — explain that. Both series are ultimately driven by the same real-world fare movements, and our route weighting is explicitly built from the same DGCA traffic data used for validation — so high correlation is expected by construction, not a red flag on its own. The more informative independent-error metrics are MAPE and RMSE, which we report for exactly that reason.

Q40. Is there data leakage risk? No — the index computation pipeline never uses DGCA benchmark fare data as an input, only as a validation target. Route/window weights are derived from a *separate* DGCA traffic-volume dataset, distinct from the benchmark fare figures used for backtesting.

Q40b. If a judge asks you to run the backtest live right now, will it reproduce exactly $r=0.9997$? The correlation, RMSE, and MAPE are computed fresh every time the backtest runs — they're real `scipy` /NumPy calculations against whatever daily index data exists in the database at that moment, not hardcoded numbers. What's worth knowing (and being upfront about if pressed): the function also returns a one-line "validation verdict" summary string that's fixed in code and always reads " $r = 0.9997$, $\text{MAPE} < 2.5\%$ " regardless of that run's actual computed values. So the *documented* $0.9997/2.41\%/₹197.47$ figures reflect one real run against the current 90-day dataset; a live re-run would compute genuinely fresh numbers, which should be close but aren't guaranteed to be bit-identical — and the verdict label wouldn't auto-update to show that. Better to say this plainly than have a judge discover it themselves.

Q41. How is the codebase itself tested, separate from economic accuracy? A 27-function `pytest` suite across 7 modules — `test_scrapers` (ethical safeguards), `test_cleaner` (dedup/outlier/decomposition correctness), `test_index_engine` (Two-Tier Jevons-Laspeyres calculation correctness), `test_backtest` (DGCA comparison logic), `test_api` (endpoint schemas), `test_esankhyiki` (export/augmentation formatting), and `test_watchdog` (tier-classification logic, including an empty-input edge case).

8. DGCA Price-Ceiling & Festive Surge Watchdog (*new — not in the uploaded PDF*)

Q42. What does the watchdog module do? Classifies every incoming fare quote into one of four regulatory tiers: **Compliant**, **Elevated Vigil** ($\geq 2.0\times$ the route's baseline fare), **Predatory Surge** ($\geq 2.5\times$, the internal threshold matching a DGCA Tariff Monitoring Unit review trigger), or **Statutory Cap Breach** (fare exceeds a defined advisory ceiling for that

corridor's distance band).

Q43. How are the corridor ceilings determined? Each of the 30 corridors is assigned a distance band (Band A: <500km up to Band F: 1600–2100km) with a corresponding advisory cap — ranging roughly **₹8,000 (Band A, short-haul) to ₹18,000 (Band F, longest routes)** — plus a historical median fare, both stored per-corridor in config alongside the route weights.

Q44. Why build this into a CPI-index project — isn't that scope creep? It's a natural extension of the same pipeline: once you're capturing clean, decomposed, route-level fares in real time, checking them against known price-sensitivity thresholds is a few extra rules on data you're already collecting, not a separate pipeline. It also gives the project a second, distinct government stakeholder (DGCA's enforcement arm) beyond NSO/RBI.

Q45. Why track "festive windows" specifically? Fare-gouging scrutiny in India clusters around specific high-demand periods — Diwali, Durga Puja, Pongal, and year-end holidays. Each window is now mapped to its actual affected corridors from the 30-route basket (e.g., the Durga Puja window covers 5 corridors into Kolkata/Patna; the year-end window covers Goa/Srinagar leisure routes plus BOM-DEL/BLR-BOM), letting the watchdog and a human reviewer apply extra scrutiny precisely when it matters.

Q46. Are the 2.0x/2.5x thresholds arbitrary? They're configured, tunable constants (`ELEVATED_SURGE_RATIO = 2.0` , `PREDATORY_SURGE_RATIO = 2.5`), not hardcoded logic — deliberately, since the "right" threshold is a regulatory decision DGCA's TMU should own, not an engineering one we should hardcode.

9. API & Dashboard

Q47. What does the API expose — full list? 12 endpoints under `/api/v1/` , grouped by tag: **System** (`/` , `/health`), **Airfare Price Index** (`/index/latest` , `/index/history`), **Metadata & Weights** (`/routes`), **Analytics** (`/lead-time-curve` , `/decomposition`), **Econometric Validation** (`/backtest`), **eSankhyiki Integration** (`/esankhyiki/augmentation` , `/esankhyiki/export`), **Scraping Engine** (`POST /scraper/trigger`), and **DGCA Price Ceiling Watchdog** (`/watchdog/summary`).

Q48. Why FastAPI specifically? Automatic interactive Swagger/OpenAPI docs, built-in request/response validation via Pydantic, native async support (useful since scraping and DB I/O are naturally async workloads), and lower latency than comparable Flask/Django setups for a data-serving API — all relevant if another government system consumes this as a live feed.

Q49. What can a user do on the dashboard? A 9-tab Streamlit app: Daily Trajectory,

MoSPI vs APIx comparison, Corridors & Map, Dynamic Pricing, **DGCA Watchdog**, Statutory Taxes, DGCA Backtest, Ethical Scraper status, and MoSPI & RBI Export — including a drill-down table of real per-route annual/daily passenger volumes and observed fares.

Q50. What does the lead-time curve show? How average fare changes as a function of booking lead time (T+1 through T+45) — a visual representation of the revenue-management pricing curve, useful both as a sanity check on scraped data and as a standalone analyst insight.

Q51. Why expose a manual `POST /scraper/trigger` if scraping runs on a schedule?

`scheduler.py` handles routine automated runs, but a manual trigger is useful for on-demand refreshes — e.g. during a live demo, or forcing an immediate re-scrape after fixing a broken scraper — without restarting the whole scheduling service.

10. Novelty, Feasibility & Comparison

Q52. What's genuinely novel compared to Google Flights / Skyscanner? Those tools help one consumer find a cheap ticket right now. APIx produces a statistically weighted, DGCA-traffic-representative, econometrically rigorous, backtested index intended for macroeconomic measurement and regulatory monitoring — and it's the only one of the two that attempts to reconcile with the actual government CPI computation pipeline via eSankhyiki, or to double as a regulatory-compliance tool via the watchdog.

Q53. Is there international precedent for web-scraped price indices? Yes — this general approach ("scanner data" / web-scraped price indices) has been explored by statistical agencies internationally for categories like electronics or apparel. APIx applies the same philosophy to Indian domestic air travel, a category well-suited to it given how digitized and dynamically priced the market already is.

Q54. What makes this feasible to build and extend? The modular architecture let each layer be built and tested independently; the index formula was mathematically well-established once the cleaned pipeline existed; the 27-test suite gave confidence at each integration step; and the synthetic engine ensured continuity even when live scraping hit friction. Extending the basket from 6 to 30 routes required no redesign — just config and weights changes, proving the architecture's extensibility in practice, not just in theory.

Q55. What's the cost/resource feasibility of a real deployment? The stack is entirely open-source (Python, FastAPI, Streamlit, SQLite/Postgres, Playwright) with no licensing cost; the main recurring cost at scale is compute for scheduled scraping and hosting, modest relative to the value of a monthly government statistical exercise, and containerizable on standard cloud infrastructure.

11. Challenges Faced

Q56. What was the hardest technical problem? Reconciling a daily, empirically-weighted market index with a monthly, differently-weighted official CPI series without producing an apples-to-oranges number — this required the eSankhyiki module to algebraically isolate and rescale components rather than naively swapping in numbers.

Q57. What was the hardest data problem? Cleaning: a naive outlier filter would wrongly flag genuinely expensive long-haul fares as errors, and naive sold-out handling (zero-filling) would silently corrupt the whole index's average. Getting the stratified IQR filtering and 85th-percentile sold-out imputation right — and getting deduplication to correctly prioritize direct-airline quotes over OTA re-listings — took real care.

Q58. What would break in production that didn't show up in your 90-day test?

Structural shifts — an airline exiting a route, a new low-cost carrier disrupting a route's price distribution, or DGCA revising its own weighting methodology — would require periodic basket/weight recalibration, similar to how official CPI baskets are periodically revised. The current pipeline computes weights from a fixed DGCA snapshot rather than continuously updating them.

Q59. How big a risk are anti-scraping measures long-term? The most operationally fragile part of the system, since it depends on external sites' cooperation. Mitigated by `base_scraper.py`'s robots.txt/rate-limit compliance (avoiding triggering blocks in the first place) and the synthetic fallback for continuity — but a production deployment would ideally also pursue official data-sharing partnerships with airlines/OTAs rather than relying purely on scraping.

12. Limitations & Future Scope

Q60. What are the honest current limitations?

- Even at 30 corridors, the basket is ~69% of national traffic, not exhaustive.
- **Live scraping is the most significant current gap:** only the airline scraper attempts a real HTTP request (to a plausible-but-unconfirmed endpoint), OTA and Playwright paths are synthetic-only today, and neither `playwright` nor `scrapy` are installed dependencies yet. The ethical guardrails are fully real; full live-data coverage is the next milestone.
- SQLite is sufficient at current scale but not built for national-scale ingestion volumes.

- Route/window weights are a fixed DGCA snapshot, not continuously updated.
- Watchdog surge/ceiling thresholds are configured defaults, not values independently validated by DGCA.
- Posted online fares aren't a perfect proxy for actually transacted fares.

Q61. What would you build next? Top priority: **wire up genuine live data collection** — enable the Playwright browser-automation path (currently commented out), give the OTA scraper a real first-attempt HTTP path (it currently goes straight to synthetic), and add `playwright / scrapy` to `requirements.txt` so they're actual runtime dependencies, not scaffolding. After that: continue expanding basket coverage toward the remaining ~31% of traffic; migrate to PostgreSQL with proper indexing for national-scale ingestion; pursue official data partnerships with airlines/OTAs as a more durable alternative to scraping; build automated periodic weight recalibration; work with DGCA to validate/calibrate the watchdog's thresholds against real TMU review criteria; and pursue a formal validation exercise with MoSPI/NSO against their internal accuracy standards.

Q62. Could this approach generalize beyond airfare? The overall pattern — scrape → clean/decompose → weight and index → validate against an official benchmark → augment the official series — is fairly general and could, in principle, apply to other highly dynamic, digitally-traded CPI components (e.g. certain e-commerce categories), though each would need its own domain-specific cleaning and weighting logic analogous to what we built here.

13. Quick-Fire Numbers (memorize these — corrected)

Metric	Value
Production index formula	Two-Tier Jevons-Laspeyres Hybrid Index (one champion formula; Paasche/Fisher/pure-Jevons are deprecated legacy stubs, not active)
Route basket	30 domestic corridors (~69% of national traffic)
Top route weight	BOM–DEL, 11.51%
Airports covered	18 (original 6 + 12 newly added: AMD, GOI, SXR, GAU, COK, PNQ, LKO, PAT, BBI, IXC, JAI, VNS)
Advance-purchase windows	T+1, T+7, T+15, T+30, T+45
Peak window	

weight	T+15, 30%
Backtest window	90 days
DGCA benchmark comparison points	18 (6 named routes × 3 monthly snapshots, Jun–Aug 2026)
Pearson correlation (r)	0.9997 (threshold: ≥ 0.88)
MAPE	2.41% (threshold: $< 5.0\%$)
RMSE	₹197.47 (threshold: $< ₹450.00$)
Reporting lead advantage	+45 days vs. official CPI
Automated tests	27 (pytest, across 7 modules)
API endpoints	12
Dashboard tabs	9
Airlines scraped	5 (IndiGo 62%, Air India 14%, Air India Express 8%, Akasa 5%, SpiceJet 4%)
OTAs scraped	5 (MakeMyTrip, EaseMyTrip, Yatra, Cleartrip, Ixigo)
GST slabs applied	5% (economy), 12% (business/premium)
Transport CPI weight	8.59% of urban CPI
Airfare headline CPI weight	~0.41%
Watchdog surge tiers	Compliant / Elevated ($\geq 2.0x$) / Predatory ($\geq 2.5x$) / Cap Breach
Festive windows tracked	4 (Diwali, Durga Puja, Pongal, year-end)

14. If They Try to Trip You Up

Q63. "Isn't this just scraping — where's the actual research/innovation?" Scraping is the data-collection layer, not the contribution. The real work is: (a) the econometric two-tier index construction using DGCA-derived empirical weights across route and booking-window dimensions, deliberately choosing Jevons at the elementary level and Laspeyres at aggregation rather than a naive single formula; (b) the cleaning methodology (stratified outlier detection, non-zero sold-out imputation, direct-airline dedup priority) tailored to airfare's specific pricing dynamics; (c) the eSankhyiki augmentation logic that correctly and algebraically reconciles a new high-frequency index with the existing official CPI composite; and (d) the watchdog's regulatory tier classification — none of which are solved by scraping alone.

Q64. "Your MAPE is 2.41% — is that good enough for a policy-grade statistic?" For a prototype validated against a proxy benchmark (DGCA route/traffic data, not the exact CPI airfare methodology) over a 90-day window across a 30-route basket, this is a strong proof of concept. Whether it clears MoSPI/NSO's own accuracy bar is ultimately their call — our contribution is demonstrating the methodology is viable and quantifying its current error margin transparently, not claiming it's production-certified.

Q65. "Why should the government trust a hackathon project's index over their own methodology?" We're not proposing to replace MoSPI's methodology — the eSankhyiki module is explicitly designed to augment the official series, keeping every other sub-component and the COICOP weighting structure untouched, only substituting a more current, higher-frequency airfare figure. It's an evidence-based supplement validated against DGCA's own data, not a wholesale replacement of statistical authority.

Q66. "You said 6 routes earlier and now say 30 — which is it?" Be upfront: the project started with a 6-route pilot basket and was deliberately expanded to 30 corridors (~69% of national traffic) as the architecture proved it could scale — route/weight expansion required no redesign of the scraper, cleaner, or index engine, only config changes, which is itself evidence the system was built to extend from day one.

Q67. "What if two team members give different answers about how a formula works?" *(Coordinate with your team beforehand.)* Every team member should be able to explain, in their own words: (1) why Jevons is used at the elementary level and Laspeyres at aggregation, and why Paasche/Fisher are deprecated rather than actively used; (2) why route + window weighting matters and where the weights come from; (3) what the eSankhyiki substitution actually changes in the CPI computation; and (4) how the watchdog's four tiers are triggered. These four are the most likely to be probed from multiple angles by different examiners.

Golden rule for the viva: if asked something not covered by your actual working code, it's

always safer to say "that's a planned extension we haven't implemented yet" than to overclaim capability the codebase doesn't have. Examiners often probe specifically to see whether a team overstates its own system — this document was deliberately fact-checked against your code for that exact reason.